



An Empirical Study of Deep Transfer Learning-Based Program Repair for Kotlin Projects

Misoo Kim*
Institute of Software
Convergence,
Sungkyunkwan University
Suwon, Korea
misoo12@skku.edu

Jinseok Heo*
Department of Electrical and
Computer Engineering,
Sungkyunkwan University
Suwon, Korea
mrhjs225@skku.edu

Youngkyoung Kim*
Department of Electrical and
Computer Engineering,
Sungkyunkwan University
Suwon, Korea
agnes66@skku.edu

Sungoh Kim
S/W Engineering Group,
Mobile eXperience,
Samsung Electronics
Suwon, Korea
sungoh5.kim@samsung.com

Hohyeon Jeong*
Department of Electrical and
Computer Engineering,
Sungkyunkwan University
Suwon, Korea
jeonghh89@skku.edu

Hyunhee Chung
S/W Engineering Group,
Mobile eXperience,
Samsung Electronics
Suwon, Korea
milou@samsung.com

Eunseok Lee[†]
College of Computing and
Informatics,
Sungkyunkwan University
Suwon, Korea
lees@skku.edu

ABSTRACT

Deep learning-based automated program repair (DL-APR) can automatically fix software bugs and has received significant attention in the industry because of its potential to significantly reduce software development and maintenance costs. The Samsung mobile experience (MX) team is currently switching from Java to Kotlin projects. This study reviews the application of DL-APR, which automatically fixes defects that arise during this switching process; however, the shortage of Kotlin defect-fixing datasets in Samsung MX team precludes us from fully utilizing the power of deep learning. Therefore, strategies are needed to effectively reuse the pre-trained DL-APR model. This demand can be met using the Kotlin defect-fixing datasets constructed from industrial and open-source repositories, and transfer learning.

This study aims to validate the performance of the pretrained DL-APR model in fixing defects in the Samsung Kotlin projects, then improve its performance by applying transfer learning. We show that transfer learning with open source and industrial Kotlin defect-fixing datasets can improve the defect-fixing performance of

the existing DL-APR by 307%. Furthermore, we confirmed that the performance was improved by 532% compared with the baseline DL-APR model as a result of transferring the knowledge of an industrial (non-defect) bug-fixing dataset. We also discovered that the embedded vectors and overlapping code tokens of the code-change pairs are valuable features for selecting useful knowledge transfer instances by improving the performance of APR models by up to 696%. Our study demonstrates the possibility of applying transfer learning to practitioners who review the application of DL-APR to industrial software.

CCS CONCEPTS

• **Software and its engineering** → *Maintaining software.*

KEYWORDS

Empirical study, Deep learning-based program repair, Transfer learning, Industrial Kotlin project, SonarQube defects

ACM Reference Format:

Misoo Kim, Youngkyoung Kim, Hohyeon Jeong, Jinseok Heo, Sungoh Kim, Hyunhee Chung, and Eunseok Lee. 2022. An Empirical Study of Deep Transfer Learning-Based Program Repair for Kotlin Projects. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*, November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3540250.3558967>

1 INTRODUCTION

Automatic program repair (APR) is a technique that automatically fixes software bugs and defects and can save significant software debugging time and costs depending on its accuracy [29]. The

* These authors contributed equally to this work.

[†] Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore

© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9413-0/22/11...\$15.00
<https://doi.org/10.1145/3540250.3558967>

interest in APR is growing substantially owing to its potential to automate software debugging tasks that directly impact cost and revenue in the industry [39]. APR performance is generally evaluated by the number of bugs or defects it fixes (i.e., the number of bugs or defects for which it generates plausible patches [50]) and how accurately fixed (i.e., the number of patches that are same as developer patches [6]) using benchmarks constructed from open-source software (OSS) projects. APR techniques can be mainly classified into template-based and deep learning-based approaches, both of which utilize past bug-patch code pairs. The template-based approach is limited by the availability of predefined templates for fixing both bugs and defects. Recently, deep learning-based APR (DL-APR) techniques that do not require predefined templates have been studied [28].

Currently, the Samsung mobile experience (MX) team is switching from Java-based projects to Kotlin-based projects in alignment with the trend shown by global IT companies [36]. We also strive to replace Java with Kotlin as our primary application-development programming language. Accordingly, we must reduce the burden of the debugging for Kotlin defects* that arise during the switching process. Simultaneously, the application of DL-APR is considered a suitable strategy for the effective utilization of large amounts of data (e.g., defect-patch code pairs) that will continuously accumulate over time. In this context, we aim to apply DL-APR to fix the defects that occur in our industrial Kotlin projects†.

Although Facebook (Meta) has proposed an APR technique focusing on specific bugs and has evaluated its performance for applications [5, 34], only a few studies have evaluated the performance of DL-APR applied to industrial software. Furthermore, many DL-APR techniques have been validated in open-source software developed in Java, C, Python, and JavaScript; however, there have been no reports of the application of DL-APR on the Kotlin defects. Since it is not possible to perfectly predict the performance of the current DL-APR techniques applied to the Kotlin projects of the MX team, we must first validate the DL-APR performance on our Kotlin defects.

The previously released DL-APR code and a pretrained model can be reused to validate the performance of the DL-APR model by either 1) training from scratch with the Kotlin defect-fixing dataset by reusing previously released DL-APR codes or 2) using the pretrained DL-APR model. However, as we are currently transforming Kotlin projects from Java projects, Kotlin defect-patch code pairs are insufficient for the first approach. Approximately 1,961 defect-fixing code pairs are available from our industrial Kotlin projects; however, they are insufficient for learning compared to the 100,000 to 14 millions bug-fixing pair datasets used in existing DL-APR studies [6, 31]. The lack of sufficient training data thus indicates that the second approach is better in the current stage.

The criteria for choosing the DL-APR model to reuse to achieve our goal are determined by whether 1) the technique is state-of-the-art (SOTA), 2) all code and data are public, and 3) their problem domain is identical to ours. The best model for our purpose is TFix [6] which generates patches for coding errors (i.e., defects) detected from a static analysis tool (SAT). More importantly, alike

our purposes, TFix aims to solve the defects detected by a SAT; therefore, TFix is the SOTA DL-APR model that best meets our criteria.

For effective reuse of TFix, two critical issues must be considered. **The first is the difference between the target language (Kotlin) and the trained language (JavaScript).** The TFix model is trained to fix JavaScript defects; however, we intend to patch Kotlin defects. **The second issue is the difference in defect types between the trained and target datasets.** TFix uses ESLint[11], but we aim to fix the defects detected by SonarQube[45]. The differences in programming languages and defect types between the training and target datasets make it difficult to guarantee the performance of TFix on our Kotlin defects.

Transfer learning can alleviate these problems. The Kotlin defect-fixing dataset collected by SonarQube's defect detection rules in industrial and open Kotlin software contains sufficient knowledge to fix SonarQube's defects detected in the Kotlin projects. Transfer learning by tuning the TFix model with this dataset can transform TFix's ability from fixing JavaScript defects to fixing Kotlin defects. Furthermore, a better performance is expected if we can utilize an additional industrial dataset (e.g., the Kotlin bug-fixing dataset‡) to supplement the domain knowledge of industrial projects and general Kotlin defect-fixing patterns.

This study aims to effectively apply DL-APR to industrial Kotlin projects. We validate the performance of TFix in fixing the defects in the Samsung Kotlin projects, then improve its performance utilizing defect- and bug-fixing datasets and transfer learning. To this end, we establish and answer three research questions:

- (1)(a) **RQ1.a. How many industrial defects can be fixed by existing DL-APR?** The TFix model can fix 94 out of 1,961 industrial defects.
- (b) **RQ1.b. Are defect-fixing datasets and transfer learning effective in improving DL-APR performance?** The TFix model tuned with industrial and OSS defect-fixing datasets can correctly fix 289 (+307%) more defects than the TFix model, establishing the fact that transfer learning is effective in improving the DL-APR performance.
- (2) **RQ2. Can an industrial bug-fixing dataset transfer useful knowledge to improve the DL-APR for defect fixing?** The model with additional transfers performed using the industrial bug-fixing dataset fixed 211 (+55%) more defects than the model transferred using only defect-fixing knowledge.

The contributions of this study are as follows.

- This study is the first to validate DL-APR's performance in fixing Kotlin defects. We show that DL-APR can correct Kotlin defects using the state-of-the-art technique, TFix.
- To the best of our knowledge, this study is the first to apply TFix to an industrial software project. Because APR studies in the context of industrial defects are insufficient, our results are valuable. We demonstrate the effectiveness of transfer learning using industrial datasets for practitioners reviewing the application of DL-APR in the field.

*We defined defects as faults detected through rules of the static analysis tool.

†We target all Kotlin applications, including android applications

‡In this study, bugs and defects are distinguished. The bug is a fault that causes unexpected behavior in software. The defect is defined as faults detected through a SAT.

- Our experiments with 19,166 OSS and 1,961 industrial Kotlin defects and 60,156 industrial Kotlin bugs showed that the DL-APR accuracy can be improved by up to 532% ($\approx \frac{594-94}{94}$) with the implementation of transfer learning.
- When transferring additional knowledge, we analyzed two aspects of the industrial Kotlin bug-fixing dataset: embedding vectors, and code ingredients. In particular, code token-based instance selection was able to fix 696% more defects compared to the initial DL-APR. The analysis results suggest an effective industrial bug-fixing instance selection direction when applying the DL-APR and transfer learning.

2 PROBLEM CONTEXT AND MOTIVATION

2.1 Java and Kotlin Projects at Samsung

Since Google announced in 2019 that Android development will become increasingly dominated by Kotlin[19], there has been a trend to switch Java projects to Kotlin [36]. Given Kotlin's valuable characteristics, such as rapid compilation, small code chunks, the safety of the null pointer exception, and its 100% compatibility with Java, there have been increased attempts to switch existing projects from Java to Kotlin [36, 48].

This has been an inexorable trend even in the IT industry. In Google, many Java projects have already been converted to Kotlin or are in the process of conversion [36]. The Samsung MX team is also working on this transformation. We investigated the commits from January 2018 to January 2022 of 140 industrial Kotlin projects and found that the number of modified Kotlin files exceeded 50% of the total monthly modified Java and Kotlin files. This trend is illustrated in Figure 1.

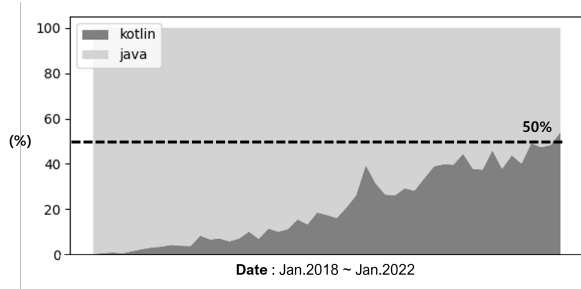


Figure 1: The trend of switching from Java to Kotlin (y-axis: accumulated ratio of committed files)

According to this trend, the Kotlin codes will increase in industrial projects in the future, inevitably increasing debugging costs. Considering these circumstances, this study intends to validate the performance of the DL-APR in industrial Kotlin projects.

2.2 Software Development with Static Analysis Tools at Samsung MX Team

The MX team applies a SAT in three steps to improve software quality within the software development process. The first is the development environment for individual developers within the team, where a SAT is applied as a plugin to each developer's local integrated development environment, and developers fix the defects

detected by the SAT. Samsung MX team uses SonarQube[45], a SAT used by our company and other companies worldwide with which many studies have been conducted [10, 12, 27, 32, 35, 46]. The second step is a continuous integration pipeline (e.g., GitHub [13], CircleCI [1] and Jenkins [2]). When the source code written by the developers of the team is committed, the SAT in the pipeline detects defects in the code. A defect report is used to evaluate the code quality of the software development team. Team leaders may triage the defect report to developers for fixing defects detected by the SAT. The third step is the quality evaluation of final software products prior to market release, in which the development team decides whether they will fix every defect detected by the SAT.

In this study, we aim to apply DL-APR to fix the defects detected by SonarQube in the first step. When defects in the first step result in unaddressed debt, higher costs are incurred in the second and third steps, respectively.

2.3 Target DL-APR: TFix

TFix is a DL-APR technique that generates patches for coding errors ("defects" in our study) detected by SATs by utilizing the Text-to-Text Transfer Transformer (T5), a powerful model that can generalize various text-to-text natural language processing tasks [6]. The authors proposed a transfer between natural and programming languages and multiple defect types to adapt the T5 model for fixing software defects. To achieve this, they built a defect-fixing dataset with defect-patch code pairs detected by ESLint, a JavaScript SAT, and then tuned the T5 model with this dataset to train an APR model by applying transfer learning.

The TFix approach can be applied to any defect written in any language and detected by any SAT. SAT defect reports indicate that this approach generates four pieces of textual information pertaining to each defect: 1) defect type, 2) defect message, 3) defect line, and 4) defect context, and uses these to train the DL-APR model from the T5 model. Figure 2 shows an example[§] of a defect-patch code pair and TFix's corresponding input template.

Input Template	fix defect type defect message defect line : defect context
Input	fix no-extra-semi Unnecessary semicolon ;;; : ;}; module.exports = HeaderH1Format; };
Output	}; module.exports = HeaderH1Format;

Figure 2: An example of instance generation of TFix

Strings with token sequences in the defect code and defect reports were matched with the input data by matching the metadata provided by the input templates (see the Input template and Input rows in Figure 2). The output of TFix is the code context in which the defect is fixed, as illustrated by the output row in Figure 2. The code printed in blue represents the fixed code line. This input/output pair is used as a single instance for transfer learning from the T5 model to the APR model.

[§]<https://github.com/quailjs/quail/commit/62e544b9e>

3 EXPERIMENTAL SETTING

3.1 Research Questions

We establish three research questions (RQs): The first validates the performance of the DL-APR applied to our industrial Kotlin projects (RQ1.a) and confirms the performance improvement through transfer learning with a Kotlin defect-fixing dataset (RQ1.b). The second analyzes whether implementing an additional industrial dataset (e.g., a bug-fixing dataset) can be effective for additional transfer learning using the defect-fixing model.

- (1)(a) RQ1.a. How many industrial defects can be fixed by existing DL-APR?
- (b) RQ1.b. Are defect-fixing datasets and transfer learning effective in improving DL-APR performance?
- (2) RQ2. Can an industrial bug-fixing dataset transfer useful knowledge to improve the DL-APR for defect fixing?

Figure 3 summarizes the input datasets and models used to answer the RQs. The datasets are collected through the process described in Section 3.2.1. The detailed methods for answering the RQs are described in Sections 4 and 5.

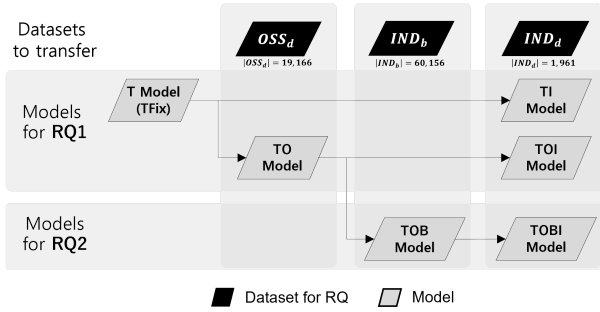


Figure 3: Inputs for answering research questions

3.2 Experimental Dataset

3.2.1 Construction Process. Figure 4 presents an overview of dataset construction process for our experiment. First, we selected 140 projects developed in Kotlin from our industrial software repository and 5,267 Kotlin OSS projects from GitHub using SEART [9]. The selection criteria for OSS projects were ten or more stars, the existence of one or more issues, and the date of the latest commit should be after January 1, 2021. OSS projects were selected on April 20, 2022.

Second, buggy commits were collected to build a bug-fixing dataset based on a search for keywords among all the commits in the selected projects. A commit was considered to be buggy when the commit message contained the bug-related keywords but did not contain the anti-keywords [31]. We filtered out commits that had more than one parent commit to avoid collecting duplicated instances [6]. Since our research goal is Kotlin defect fixing, the bug-patch code pairs are collected only from Kotlin source files. We converted the set of bug-patch code pairs from the selected commits into TFix training instances as described in Section 2.3. For the bug-fixing dataset, the defect type and message of bug-patch code pairs required for the TFix input could not be identified because these

bugs were not detected by a SAT; therefore, we left the metadata string blank.

Third, we used SonarQube to mine the defect-fixing dataset from the bug-fixing dataset. After performing static analysis, we compared the defect list with each Kotlin source file before and after modification, and checked whether the detected defects no longer appeared in the modified patch code. 19,166 (OSS_d) and 1,961 (IND_d) defect-fixing code pairs were collected from the OSS and industrial Kotlin projects, respectively, as tuning instances for TFix. SonarQube returns the rule ID (e.g., S100), whereas ESLint returns the defect type (e.g., no-extra-semi), which is used as the input of TFix. Accordingly, the rule ID was used as the defect type to construct the input for TFix.

The bug-fixing dataset collected in the second phase includes a defect-fixing dataset. We excluded 1,961 defect-fixing code pairs from the bug-fixing dataset because some ground-truth instances of the defect-fixing code pairs could be trained when tuning with the bug-fixing dataset in the experiment for RQ2. Consequently, we selected 60,315 bug-fixing code pairs (IND_b) as the tuning dataset for RQ2.

3.2.2 Dataset Statistics. Table 1 is a statistics of dataset obtained from the OSS and industrial Kotlin projects. Among the 62 Kotlin defect detection rules provided by SonarQube by default, 32 rules detected 19,166 defects in the OSS dataset, and 16 rules detected 1,961 defects in the industrial dataset. These defects were classified as code smell, bug[†], or security hotspot (hot) classes, and their severity was classified as critical, major, minor, info, or blocker. Our experimental dataset consists of defects with various detection rules, types, and severities.

Although there was a difference between OSS_d and IND_d in the number of defect instances for each detection rule, the instance distribution in dominant defect rules (S1128, S1135, S1481 and S1172) were similar, potentially implying that the OSS defect-fixing dataset can complement the industrial defect-fixing dataset.

3.3 Evaluation Metrics

Accurate correctness evaluation of generated patches is difficult without human review because of the patch plausibility problem [51]. We used Exact Match (EM), a strict metric, to measure patch correctness. EM is the number of defects for which a DL-APR can generate developer patches. A prediction is considered correct only if the fixed code perfectly matches the developer's fixed code. Based on the EM, the defect-fixing ratio (FR) is computed by $\frac{\#ExactMatch}{\#Defects}$.

3.4 The TFix Model and Tuning Details

3.4.1 Pretrained Defect-Fixing Model. We used the TFix base model published by the authors as the initial pretrained defect-fixing DL-APR model. The authors provided pretrained small, base, and large TFix models with parameters of 60 million, 220 million, and 770 million, respectively, whose FRs are 39.2%, 48.5%, and 49.3%, respectively. The large model showed the highest accuracy, but its performance improvement over the base model was marginal (approximately 0.8%), despite the three-fold more parameters; therefore, we chose a scalable base model rather than a large model.

[†]This bug simply means one of the classes provided by SonarQube.

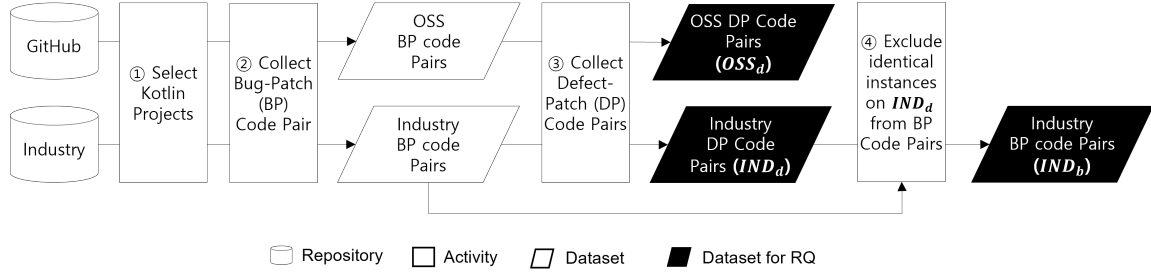


Figure 4: An overview of dataset construction process

Table 1: Experimental defect dataset

Class (#Class)	Defect Information		#Defects (#Projects)	
	RuleID	Severity	OSS	Industrial
Smell (23)	S1128	MINOR	13,367 (1572)	1,492 (50)
	S1135	INFO	1,659 (519)	140 (22)
	S1481	MINOR	1,057 (422)	71 (15)
	S1172	MAJOR	972 (348)	100 (18)
	S1874	MINOR	425 (72)	14 (6)
	S1192	CRITICAL	327 (175)	52 (8)
	S117	MINOR	245 (89)	31 (5)
	S1134	MAJOR	244 (89)	18 (6)
	S100	MINOR	163 (51)	
	S1186	CRITICAL	128 (64)	4 (3)
	S101	MINOR	81 (12)	
	S108	MAJOR	83 (54)	9 (2)
	S3776	CRITICAL	70 (61)	9 (6)
	S1144	MAJOR	57 (47)	3 (2)
	S1940	MINOR	33 (6)	
	S1125	MINOR	31 (25)	4 (3)
	S107	MAJOR	11 (9)	
	S4144	MAJOR	10 (7)	
	S1066	MAJOR	9 (9)	
	S1133	INFO	7 (5)	
	S4663	MINOR	6 (6)	
	S6318	MINOR	3 (2)	
	S1110	MAJOR	3 (3)	
Bug (6)	S1764	MAJOR	30 (25)	4 (3)
	S1656	MAJOR	24 (16)	7 (4)
	S1763	MAJOR	16 (14)	
	S1862	MAJOR	13 (8)	3 (3)
	S1145	MAJOR	11 (8)	
	S3923	MAJOR	10 (7)	
Hot (3)	S2245	CRITICAL	54 (26)	
	S1313	MINOR	14 (10)	
	S2068	BLOCKER	3 (2)	
Total	32 Rules	5 Types	19,166 (5267)	1,961 (140)

3.4.2 Tuning Process. We used two tuning datasets, the Kotlin defect-fixing dataset collected from OSS projects (OSS_d) and the Kotlin defect-fixing dataset collected from industrial projects (IND_d). Since we expected IND_d to contribute more to industrial defect

fixing than OSS_d , we first tuned with OSS_d then tuned IND_d for knowledge transfer.

3.4.3 Tuning/Testing Dataset. We used 90% of OSS_d for tuning and 10% for validation. As we wanted to verify the ability to fix industrial defects, we did not use OSS_d for testing. We performed stratified k -fold cross-validation to prevent erroneous experiments when tuning with IND_d because the tuning and test instances could be identical [43]. As shown in Table 1, the rule IDs with the fewest instances were S1144 and S1862 and there were three such instances. We set k to 3 to ensure both tuning and testing of the defect-patch code pairs detected by these rules. In summary, out of 2/3 of the IND_d , 90% was used for training (tuning) and 10% for validation, while the remaining 1/3 is used for testing. This testing was repeated three times to show performance for all defects.

3.4.4 Environments. We fine-tuned the TFix model on three GPUs (Nvidia Tesla V100s) with 200GB RAM. Tuning with OSS_d and IND_d took 191 and 15 minutes on average per fold, respectively.

4 RQ1. EFFECTIVENESS OF DL-APR AND TRANSFER LEARNING

4.1 Method

To confirm the validity of TFix and effectiveness of transfer learning, we evaluated and compared the EM and FR of patches generated using the following models:

- (1) T: The original TFix model
- (2) TO: The T model fine-tuned with OSS_d
- (3) TI: The T model fine-tuned with IND_d
- (4) TOI: The TO model fine-tuned with IND_d

The test is performed with the entire IND_d in the case of T and TO models, as mentioned in Section 3.4. In the case of the TI and TOI models, IND_d is divided 3-fold for tuning and evaluation.

4.2 Results

The experimental results are summarized in Table 2. The answers to the research questions in this chapter are based on this table.

4.2.1 RQ1.a. Performance of DL-APR. Although TFix is a model trained with JavaScript defects detected by ESLint, the T model was able to generate correct patches for 94 (4.8%) of 1,961 defects, all of which corresponded to code smells.

The correctly patched defects were those detected using rules S1128, S1172, S1481, S1134, and S1135, which could only be

Table 2: Exact match and fixing ratio (%) of TFix variations (bold: the highest value)

Class	RuleID	#All	T	TO	TI	TOI
Smell	S1128	1,492	82 (5.5)	129 (8.6)	132 (8.8)	225 (15.1)
	S1135	140	2 (1.4)	19 (13.6)	19 (13.6)	26 (18.6)
	S1172	100	7 (7.0)	55 (55.0)	46 (46.0)	66 (66.0)
	S1481	71	2 (2.8)	24 (33.8)	23 (32.4)	32 (45.1)
	S1192	52	0 (0.0)	0 (0.0)	0 (0.0)	2 (3.8)
	S117	31	0 (0.0)	3 (9.7)	16 (51.6)	19 (61.3)
	S1134	18	1 (5.6)	1 (5.6)	1 (5.6)	1 (5.6)
	S1874	14	0 (0.0)	0 (0.0)	3 (21.4)	4 (28.6)
	S108	9	0 (0.0)	5 (55.6)	4 (44.4)	5 (55.6)
	S3776	9	0 (0.0)	1 (11.1)	1 (11.1)	2 (22.2)
	S1125	4	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
	S1186	4	0 (0.0)	1 (25.0)	0 (0.0)	1 (25.0)
	S1144	3	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
	S1656	7	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
Bug	S1764	4	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
	S1862	3	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)
Sum		1,961	94 (4.8)	238 (12.1)	245 (12.5)	383 (19.5)

resolved by removing the existing code. The defects corresponding to rules S1128, S1172, and S1481 could be fixed by deleting unused import statements, function parameters, and local variables, respectively, and were similar to no-unused-vars^{||}, a defect type already learned by TFix.

Attempts at fixing defects using the T5 model^{**} were unsuccessful (zero correct patch). Considering that 4.8% of defects could be correctly fixed by TFix, TFix can be seen as a more effective APR model than T5 for Kotlin defect fixing. Our experimental results showed that knowledge of JavaScript defect-fixing patterns already learned by the TFix model could be used to fix Kotlin defects. Therefore, although the TFix model is trained on a different defect dataset with a different language and SAT than our target defects for patch generation, it can be used as a baseline model from the perspective of APR, which is the same domain.

Answer to RQ1.a.

TFix generated correct patches for 94 (4.8%) code smell defects from industrial Kotlin defects.

Finding 1.

Even if the DL-APR model is trained with defects detected in a language and SAT different from those of the target defects to be patched, a pretrained DL-APR model is more effective in fixing defects than a model from a completely different domain.

4.2.2 RQ1.b. Effectiveness of Transfer Learning. The performance rankings of the four models were TOI, TI, TO, and T (19.5%, 12.5%,

12.1%, and 4.8%, respectively). A total of 383 (19.5%) defects were corrected by the TOI model with transferred knowledge from both OSS_d and IND_d , so the TOI model fixed 307%^{††} more defects than the T model. The TO and TI models generated patches for 153% and 161% more defects, respectively, compared to the T model, indicating that transfer learning can improve APR performance by generating more correct patches than by using TFix alone.

Performance of the TO model: The TO model fixed 238 defects (12%), including the same defect types detected by rules S1128, S1172, S1481, S1134 and S1135 for which the T model succeeded in generating a patch. In addition, the TO model fixed four more code smell defects detected by rules, S108, S117, S1186 and S3776, which the T model could not fix. These results show that fine-tuning the model with OSS_d can transfer defect-fixing knowledge (i.e., the ability to fix the defects detected by rules S108, S117, S1186 and S3776), which lack in the TFix model, to the existing APR model. Therefore, fine-tuning the base DL-APR model with the OSS defect-fixing dataset raises the chance to fix these defects.

Performance of the TI model: The TI model generated correct patches for 221 defects detected by the five common types of rules fixed by the T model, further fixing four more types of code smell defects that the T model could not fix. Similar to why OSS_d was effective in transfer learning, the reason for showing these results is because the defect-fixing knowledge that could not be learned from JavaScript defects was in IND_d . Therefore, IND_d can also help transfer knowledge of Kotlin defects to APR.

Comparison between the TO and TI models: The TO model could fix defects detected by rule S1186, whereas the TI model could not. IND_d had only 4 defect instances detected by rule S1186. This capability seems to have been fixed in the TO model because defects detected by S1186 were lacking in IND_d . On the other hand, the TI model could resolve defects detected by S1874, which could not be fixed by either T or TO. The rule S1874 detects a defect that uses deprecated code (e.g., a function, variable, etc.) which can be resolved by replacing the deprecated code with other code. In other words, tuned DL-APR could fix this defect when the pair of token A to be replaced and token B which must be replaced both existed in the tuning dataset. Since these ingredient codes such as function and variable names depend highly on the software project [55], the industrial dataset can obviously be much more effective in detecting them than the OSS dataset. As in the example above, some defects are highly dependent on the ingredient codes that implicitly contain industrial domain knowledge; even though IND_d is quantitatively smaller than OSS_d , it is bound to contain better information for fixing industrial defects.

Performance of the TOI model: The TOI model contains knowledge of both OSS_d and IND_d . Since the advantages of the TO and TI models over the T model mentioned above are combined in the TOI model, this model was able to correct most defects. TOI was able to create patches for 383 (19.5%) of the total defects and correct 11 out of 16 types of defects. The TOI model was able to fix the defects detected by all the rules that the T, TO, and TI models could fix, and the TOI model was able to fix the same number or more defects for each type. Furthermore, the TOI model fixed defects detected by the rule S1192 which the other models could not.

^{||}<https://eslint.org/docs/rules/no-unused-vars>

^{**}<https://huggingface.co/t5-base>

^{††}Subsequently, the improvement rate is computed by $\frac{NewEM - PreviousEM}{PreviousEM}$

Defects detected by the rule S1192 are critical code smells which can cause code maintenance problems because they repeatedly use a duplicate string. We expect that a patch pattern that can correct this defect could be learned by accumulating knowledge from both OSS_d and IND_d . Comparing the results of verifying the effects of transfer learning by different datasets, the TOI model can generate patches for 61% and 56% more defects than the TO and TI models, respectively.

Answer to RQ1.b.

The models transferred from the OSS and industrial defect-fixing datasets were able to solve 144 and 151 more defects than the TFix model, respectively, and 289 more defects were resolved when the knowledge of both datasets was transferred. Therefore, transfer learning with OSS and industrial defect-fixing datasets is effective for industrial defect-fixing APR.

Finding 2.

The fine-tuned model using OSS and industrial defect-fixing datasets fixed the most defects and the most rules. In particular, this model corrected new defect rules that could not be corrected by the APR model tuned using the respective datasets.

5 RQ2. IMPACT OF BUG FIXING DATASETS ON TRANSFER LEARNING

5.1 Method

We confirmed that the TOI model surpassed the others in fixing industrial Kotlin defects. In addition, we confirmed that the TI model's industrial domain knowledge (e.g., ingredient codes) could improve APR performance by providing knowledge that cannot be obtained from OSS. This finding implies that the industrial dataset contributes more to patch generation than the OSS dataset. If we obtain and train an extra dataset that can supplement the knowledge of industrial projects while maintaining knowledge to fix defects from the company's accumulated dataset, we can expect a higher performance of DL-APR.

The industrial bug-fixing dataset is a feasible complementary dataset that can provide knowledge such as variable names within the industrial code. In addition, **bug fixing** is similar to **defect fixing** in terms of modifying the source code to a "better code". AVATAR showed that SAT defect-fixing patterns could contribute to general semantic bug-fixing [30], expecting that the reverse is also possible. Accordingly, we must investigate whether an industrial Kotlin bug-fixing dataset can be used as a complementary dataset for Kotlin defect-fixing with transfer learning.

To determine the effectiveness of the industrial bug-fixing dataset (IND_b) comprising 60,315 bug fixing instances described in Section 3.2.2 as a transfer learning dataset for defect-fixing APR, we built the TOBI model by tuning the TO model using IND_b prior to tuning with IND_d .

5.2 Results

The TOBI column of Table 3 shows the experimental results answering RQ2.

Table 3: Exact match of Tuning dataset variations (bold: higher than TOI, gray box: the highest value)

Type	RuleID	#All	TOI	TOBI				
				default	Clustering balance	real	CodeToken replace	select
Smell	S1128	1,492	225	368	272	393	411	471
	S1135	140	26	50	34	46	55	57
	S1172	100	66	68	51	74	76	84
	S1481	71	32	36	30	36	38	42
	S1192	52	2	26	11	25	19	31
	S117	31	19	23	22	22	19	25
	S1134	18	1	2	2	4	4	9
	S1874	14	4	7	5	8	8	9
	S108	9	5	4	5	4	3	4
	S3776	9	2	4	3	3	3	4
	S1125	4	0	3	0	3	2	3
	S1186	4	1	0	0	0	0	0
	S1144	3	0	0	0	0	0	1
	S1656	7	0	0	0	1	1	3
Bug	S1764	4	0	3	0	1	0	2
	S1862	3	0	0	0	0	0	3
Sum		1,961	383	594	435	620	639	748

First, we compared the performance of the TOI model tuned only with the defect-fixing dataset, and then the TOBI model was further tuned with the bug-fixing dataset. As a result of the experiment shown in Table 3, TOBI fixed a total of 594 defects, 211 more than that of the TOI model. The TOBI model can resolve the defects detected by all rules that the TOI can fix, except for rule S1186 (see the Section 7.2). The number of fixed defects was also greater or equal. Furthermore, the TOBI model can fix new types of defects, including bug-type defects detected by rule S1764 and code smell defects detected by rule S1125. These results confirm that the industrial bug-fixing dataset effectively transfers knowledge of fixing industrial defects, even though it is not a defect-fixing dataset.

The defects detected by the rule S1125, which can be newly fixed, use duplicate Boolean literals (e.g., `! true` and `true is true`). These defects damage source code readability and consequently can make maintenance difficult. They can be fixed by replacing or deleting Boolean literals (e.g., `false` and `true`). Considering the number of possible changes, many defect-fixing change pairs are required, but only 31 and 4 defect instances are in OSS_d and IND_d , respectively. The defect detected by rule S1764 is a bug type that could not be fixed in the model trained with the defect-fixing dataset. This defect occurs when the expressions on both sides of the binary operator are the same (e.g., `a[0] == a[0]`), where one of the expressions must be changed. There are 30 and 4 such defect instances in OSS_d and IND_d , respectively. In other words, defects detected by rules such as S1125 and S1764 had quantitative problems in OSS_d and IND_d leading to a lack of knowledge about which ingredient codes to generate the patch. However, the bug-fixing dataset contains knowledge which can supplement these defects. In particular, the defects detected by rule S1764, which is a bug class in SonarQube, are expected to have a similar patch pattern in the bug-fixing dataset which can be regarded as a similar context.

Answer to RQ2

When the additional transfer is performed with IND_b , the model can fix 211 (+55%) more defects than the model transferred only with defect-fixing knowledge, confirming that the industrial bug-fixing dataset can transfer useful knowledge to fix industrial defects.

5.3 Discussion

5.3.1 Context. Since two datasets, IND_d and IND_b , are not for exact same task, not all instances of IND_b are helpful. If we can check which instances of IND_b can be used as effective transfer learning instances for APR, we can more effectively utilize the many bug-fixing datasets accumulated in industrial projects.

To investigate effective instances for Kotlin defect fixing, we generate and compare variations of IND_b for tuning the pretrained DL-APR model. For the variation of IND_b , we establish two assumptions for the instance selection method. The key to assumptions is that the datasets similar to IND_d , our target industrial defect-fixing dataset, are more effective transfer learning data [16].

- **Word embedding vector-based selection.** IND_b reconstructed similar to the distribution of IND_d based on the word-embedding vector is effective for transfer learning.
- **Code token-based selection.** IND_b having code tokens of IND_d is effective for transfer learning.

We established the following method to analyze the above assumptions: Our experiment was performed as a 3-fold validation as in the previous experimental setting. This prevents observation of the defect-fixing test dataset in the instance selection phase. The comparison between distributions of IND_d and IND_b is also performed with 2/3 instances of IND_d .

5.3.2 Embedding Vector-based Instance Selection. Clustering is a method used to check the distribution of features in a dataset [17, 22]. We 1) converted the text of each instance in IND_d and IND_b into word-embedding vectors, 2) performed clustering based on the vectors of instances of IND_d to build a clustering model, 3) checked the distributions of the number of instances per cluster of IND_d , and 4) inferred the cluster of IND_b instances using the clustering model. The distribution of the number of instances per cluster of IND_d is regarded as distribution of the industrial defect-fixing dataset. We reconstructed IND_b according to the distribution of IND_d .

For each instance of both IND_d and IND_b , defect code and patch code are transformed into embedding vectors with the embedding layer of the TOI model. Then, the vectors of defect code and patch code are concatenated to represent a single instance.

Next, to build the clustering model, we used the x-means clustering of the open-source library PyCluster [40]. The advantage of x-means clustering is that it automatically determines the optimal number of clusters [41]. With embedding vectors of IND_d extracted in the above step, instances are classified into x clusters with a x-means clustering algorithm. Here, the clusters and the number of instances in each cluster serve as indicators to check the distribution of the entire vector dataset.

We can determine the closest cluster of a new instance using its distance to the centroid of each cluster. To reconstruct a tuning dataset IND_b having a similar distribution with IND_d , the cluster closest to each instance of IND_b is inferred as the instance's cluster. Based on the inferred cluster of instances of IND_b , we compare the performance of the two models below to check whether the tuning data with similar distribution of IND_d is effective for transferring knowledge.

- **TOBI-balance:** Instances from IND_b are selected so that each cluster has an equal number of instances, regardless of the distribution of IND_d .
- **TOBI-real:** Instances from IND_b are selected according to the cluster-specific instance distribution of IND_d .

Results. The Clustering column in Table 3 summarizes the experimental results. TOBI-real, which is similar to the distribution of the embedding vector of IND_d , fixed 620 defects. This model can fix more defects than simply trained TOBI and TOBI-balance with uniform distribution. TOBI-balance corrected 435 defects, a performance confirmed to be worse than that of either TOBI or TOBI-real. Therefore, we demonstrated that the bug-fixing dataset similar to the embedding vector distribution in IND_d is more effective data for tuning the defect APR model on average. Table 4 lists the results for each fold.

Table 4: Comparison on cluster-based instance selection

Fold (#Clusters)	Model	Type	$ IND_b $	Distribution (ratio)	#EM(FR(%))
1 (10)	TOI	-	-	11:9:11:10:7:6:15:12:15:3	128 (19.4)
	TOBI	-	60,315	15:16:9:10:5:9:10:10:11:4	200 (30.3)
	TOBI	balance	25,960	1:1:1:1:1:1:1:1:1:1	148 (22.4)
	TOBI	real	41,553	11:9:11:10:7:6:15:12:15:3	223 (33.8)
2 (10)	TOI	-	-	12:8:5:6:15:11:13:15:14:2	130 (19.9)
	TOBI	-	60,315	9:5:5:9:18:14:13:11:11:5	208 (31.9)
	TOBI	balance	27,220	1:1:1:1:1:1:1:1:1:1	158 (24.2)
	TOBI	real	42,113	12:8:5:6:15:11:13:15:14:2	203 (31.1)
3 (8)	TOI	-	-	13:13:15:7:9:22:18:3	128 (19.3)
	TOBI	-	60,315	17:21:12:10:8:14:13:4	186 (28.7)
	TOBI	balance	26,140	1:1:1:1:1:1:1:1	129 (19.9)
	TOBI	real	40,279	13:13:15:7:9:22:18:3	194 (29.9)

There were 10, 10, and 8 clusters for each fold. For each fold, the distribution (Column Distribution (ratio)) shown in the original TOBI is different from the TOI. In other words, fitting the distribution to the TOI, named TOBI-real, shows better performance than the original TOBI and TOBI-balance on average. Ten clusters were built in the first fold, and the minimum and maximum ratios of the instances for each cluster were 3 and 15, respectively. In the first fold, TOBI-real fixed 23 (223-200) more defects than TOBI despite having fewer instances (60,315 vs. 41,553). In contrast, the TOBI-balance fixes 52 (200-148) fewer defects. TOBI-real fixed five fewer defects than TOBI in the second fold but eight more than TOBI in the third fold. Although the results were different for each fold, selecting instances from IND_b based on the embedding vectors' distribution effectively generates a tuning dataset which can increase EM while reducing computation cost of training.

Finding 3.

The procured industrial bug-fixing dataset regarding the distribution of the embedding vector can improve the defect-fixing model.

5.3.3 Code Token-based Instance Selection. An ingredient code token is a piece of code required to create a patch of the APR [53], and defect code's tokens provide the defect context. The absence of these code tokens in the training and tuning datasets leads to the out-of-vocabulary problem referred to in deep learning [15], which significantly reduces the performance of the DL-APR model [8]. The tuning dataset IND_b containing code tokens from the industrial Kotlin defect dataset IND_d may show better patch generation performance. To verify this, we evaluate the performance of a variation of IND_b generated with the tokens in IND_d .

First, to select code tokens from IND_d , we 1) selected instances' pure source code without comments, strings, or annotations, 2) tokenized the codes with NLTK Tokenizer [3], and 3) selected only tokens beginning in English. To validate our assumption, we compared the performance of the two models described below to check whether the tuning dataset containing code tokens from IND_d was suitable for knowledge transfer.

- **TOBI-replace:** Instances where IND_b 's tokens not in IND_d are replaced with similar tokens in IND_d
- **TOBI-select:** Instances from IND_b having the code tokens of IND_d

For the TOBI-replacement model, we replaced the code tokens of IND_b (target tokens) with the tokens from IND_d , which have the same token type and similar meaning. We classified tokens into three types according to the Kotlin Coding Style Guide [18]: 1) tokens beginning with a lowercase letter (e.g., function and variable names), 2) tokens beginning with an uppercase letter (e.g., class names) and 3) tokens comprised entirely of uppercase letters (e.g., constants). We considered a pair of target and replacement tokens as a convertible pair only if these tokens were of the same type. Next, we evaluated tokens' semantic similarity by calculating the distance between their vectors based on word embedding [24]. We generated the word embedding-based vector for each token from IND_b and IND_d as described in Section 5.3.2. After vectorizing the tokens, we computed the similarity between target tokens' and candidate replacement tokens' vectors by cosine similarity. Based on this similarity, we replaced target tokens of IND_b , which is not in IND_d , with the most similar tokens in IND_d .

Results. The CodeToken column in Table 3 summarizes our experimental results. The TOBI-replacement model fixed 639 defects. The TOBI-selection model fixed 748 defects, 154 and 109 more defects than TOBI and TOBI-replacement models, respectively. The TOBI-selection model's performance was the best among the various models and dataset variations.

Table 5 shows the results for each fold. The TOBI-selection model demonstrated better performance than the other models for all folds. Similar to the TOBI-real model, the TOBI-selection model performed better than the TOBI model, even with fewer instances. Therefore, selecting instances from IND_b based on the code tokens

from IND_d effectively generated a tuning dataset that increased EM while reducing the computation cost of training.

Interestingly, the TOBI-replacement model fixed 639 defects, fewer than the TOBI-selection model. We expected that word replacement would yield better performance, but this proved not to be the case. The higher performance of the TOBI-selection model implies that maintaining the structure of the surrounding code token is more effective than simply having the token in IND_d . Token type and semantic similarity were considered in our token replacement method; however, the structure of the source code and the relationship between other tokens around a token were not considered. This result implies that each code token in IND_d may depend on a specific code structure or buggy context.

Table 5: Comparison on code token-based instance selections

Fold	Model	Type	$ IND_b $	#EM (FR(%))
1	TOBI	-	60,315	200 (30.3)
	TOBI	replace	60,315	218 (33.0)
	TOBI	select	53,479	270 (40.9)
2	TOBI	-	60,315	130 (19.9)
	TOBI	replace	60,315	213 (32.7)
	TOBI	select	53,571	213 (32.7)
3	TOBI	-	60,315	125 (19.3)
	TOBI	replace	60,315	208 (32.0)
	TOBI	select	53,582	265 (40.8)

Finding 4.

The procured industrial bug-fixing dataset having code tokens from the industrial defect-fixing dataset can improve the performance of the defect-fixing model.

6 THREATS TO VALIDITY

6.1 Internal Validity

We used the same methods as the TFix authors to minimize experimental errors. We used the released code as-is, just changing the dataset path to minimize the errors of patch generation, evaluation, and tuning. Our dataset collection process was also adapted from their method. We did not evaluate error fixing from among their suggested evaluation metrics, using the more accurate metric, Exact Match instead. In the discussion section, we could have derived more accurate convertible pairs for the TOBI-replacement model by using the AST node type when replacing the code tokens [53]. Although we made this part heuristic, we tried to guarantee maximum accuracy by adopting code conventions much lighter than those of AST analysis methods.

6.2 External Validity

There are two major external liabilities pertaining to the generalization of our research. The first is that we used only TFix for DL-APR

validation and improvement, so we do not know if our results will be comparable to those of other DL-APRs [6, 14, 20, 31, 54]. However, due to the characteristics of DL, the effects of tuning-based transfer learning and the tuning datasets themselves, which we validated, are expected to result in effects similar to those of other models. Second, there are various methods for transfer learning [49, 56], and we did not use the SOTA transfer method. Furthermore, since we tuned the model by progressively accumulating different datasets, recently transferred knowledge may exert a more significant impact than knowledge accumulated in the past [21]. The current transfer strategy could be one of the causes that TOBI failed to fix S1186 in Section 5.2. The defects detected by S1186 need an action for implementing new logic in the blank function block. The same defect-fixing pair in the tuning dataset from IND_d , so the TOI model can successfully fix this defect while TOBI cannot. We expect the knowledge gained by this pair was diluted while tuning with the bug-fixing dataset, and this implies the need for applying better transfer learning methods in the future. However, we used basic transfer learning, as our goal was to validate the effectiveness of transfer learning itself. In the future, we plan to apply the SOTA transfer method, which enables more effective knowledge transfer and will likely show better results than ours.

7 RELATED WORKS

7.1 APR for Defects

APR techniques for fixing general semantic bugs use a test suite for fault localization and patch validation [29, 31], whereas APR for fixing defects uses SATs (e.g., SonarQube [45], SpotBugs [47], PMD [42], and ESLint [11]). SATs provide a defect report including defect location, defect types, descriptions, and correction examples, and sometimes recommend patches for specific defect types [7]. In short, there is a difference in that it is relatively easy to obtain a context for fixing defects compared to fixing bugs [20, 23, 52].

Patch generation approaches for fixing defects can be roughly classified into template-based and deep-learning-based methods. SpongeBugs created a fixing template using textual patterns and AST-based edit patterns for specific defect types [33]. MemFix collects code patterns about memory allocation statements in C language and uses clean code patterns, which have no memory allocation defects, as oracles and templates [26]. Some studies generated defect-fixing templates using clustering methods [5, 30, 44]. TFix [6], a DL-APR, is a method of generating a patch for a given defect code by learning a defect-correction pair with the DL model without the need to create a separate template. At this time, in the interest of the accuracy of the generated patch, the defect message and the code surrounding the defect were used as context information. To the best of our knowledge, TFix showed the best performance in automatic defect correction and was thus selected as the target APR model for our study.

7.2 APR in Industry

APR has also received considerable attention in the industry. Naitou et al. verified existing APR research in the industry, and in the process, verified that the performance of APR applied in the field is still low and that obtaining a sufficient number of test cases may not be possible [38]. Based on these findings, the authors argued

for the need for APR studies. Similarly, Noda et al. showed that the APR performance overfits the OSS dataset by applying the latest APR technique (ELIXIR) to industrial software [39]. They also mentioned that the existing APR tool assumes the existence of a failed test case that may be invalid in an industrial environment. Based on these findings, APR tools have been proposed for the industry [4, 5, 25, 34, 37].

Some studies have validated the application of template-based APR to industrial datasets. SapFix was first proposed by Facebook for large-scale industrial APR [34]. In a subsequent study, Getafix proposed a hierarchical clustering algorithm to compensate for the problem of the excessive size of the fixing-pattern search space [5]. Gunnar et al. proposed kBar based on TBar [29], and verified it using 14 real bugs from the Saab company dataset [4]. The other mainstream approach, DL-APR, has the advantage of not needing to find a template directly, and has thus attracted considerable attention from the industry recently. Google proposed a neural machine translation-based DeepDelta to correct compilation errors and demonstrated a patch success rate of approximately 50% for two types of compilation errors collected from industrial projects [37]. Most of the existing studies have been proposed and verified using traditional programming languages, such as Java. However, as mentioned in our research context (Section 2.1), existing techniques have limitations in directly applying them to a new language such as Kotlin because the trend of introducing new languages in the field is not considered. Therefore, we suggested a method to verify and improve DL-APR performance under these circumstances.

8 CONCLUSION

This study validated a pretrained DL-APR model for fixing Kotlin defects in industrial projects. We found that the defect-fixing ratio of the SOTA DL-APR model was only approximately 5% because of the absence of the SonarQube Kotlin defect-fixing dataset that we targeted. To solve this problem, we applied transfer learning with Kotlin defect-fixing datasets from industrial and OSS projects, and improved the bug-fixing ratio by 307%. Furthermore, we showed that 532% more defects could be corrected than the pretrained DL-APR model by transferring the accumulated bug-fixing knowledge from the company. In this process, we found that the embedding vectors and code tokens of the source code can be utilized as a feature to select an effective tuning instance.

Our study provides important insights for industry stakeholders by considering the application of DL-APR. Based on the pretrained DL-APR model, the defect/bug-fixing datasets accumulated by the company and through transfer learning can automatically fix defects that occur in the real world. In addition, the practitioners can more effectively utilize knowledge accumulated within their company through IDE or API based on our study. In the future, we plan to improve the defect-fixing ratio by applying state-of-the-art transfer learning to transfer knowledge of industrial data to the existing DL-APR model.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(2019R1A2C2006411, 2021R1A6A3A01086997) and Samsung Electronics.

REFERENCES

- [1] 2022. CircleCI. <https://circleci.com/> Accessed: 2022-05-19.
- [2] 2022. Jenkins. <https://www.jenkins.io/> Accessed: 2022-05-19.
- [3] 2022. NLTK Tokenizer. <https://www.nltk.org/api/nltk.tokenize.html> Accessed: 2022-05-19.
- [4] Gunnar Applelid. 2021. Evaluating template-based automatic program repair in industry.
- [5] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–27. <https://doi.org/10.1145/3360585>
- [6] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. Tfix: Learning to fix coding errors with a text-to-text transformer. In *International Conference on Machine Learning*. PMLR, 780–791.
- [7] António Carvalho, Welder Luz, Diego Marcilio, Rodrigo Bonifácio, Gustavo Pinto, and Edna Dias Canedo. 2020. C-3PR: A Bot for Fixing Static Analysis Violations via Pull Requests. *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)* 00 (2020), 161–171. <https://doi.org/10.1109/saner48275.2020.9054842>
- [8] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959. <https://doi.org/10.1109/TSE.2019.2940179>
- [9] Ozren Dabic, Emad Aghajani, and Gabriele Bavota. 2021. Sampling Projects in GitHub for MSR Studies. In *18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021*. IEEE, 560–564. <https://doi.org/10.1109/MSR52588.2021.00074>
- [10] Khashayar Etemadi, Nicolas Harrand, Simon Larsen, Haris Adzemovic, Henry Luong Phu, Ashutosh Verma, Fernanda Madeiral, Douglas Wikstrom, and Martin Monperrus. 2021. Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations. *arXiv preprint arXiv:2103.12033* (2021). <https://doi.org/10.48550/arXiv.2103.12033>
- [11] Open JS Foundation. 2022. ESLint - Pluggable JavaScript liner. <https://eslint.org/> Accessed: 2022-05-11.
- [12] Javier Garcia-Munoz, Marisol Garcia-Valls, and Julio Escribano-Barreno. 2016. Improved metrics handling in SonarQube for software quality monitoring. In *Distributed Computing and Artificial Intelligence, 13th International Conference*. Springer, 463–470. https://doi.org/10.1007/978-3-319-40162-1_50
- [13] github. 2022. GitHub. <https://github.com/> Accessed: 2022-05-19.
- [14] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. Deepfix: Fixing common c language errors by deep learning. In *Thirty-First AAAI Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v31i1.10742>
- [15] Vincent J Hellendoorn and Premkumar Devanbu. 2017. Are deep neural networks the best choice for modeling source code?. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 763–773. <https://doi.org/10.1145/3106237.3106290>
- [16] Tzu Ming Harry Hsu, Wei Yu Chen, Cheng-An Hou, Yao-Hung Hubert Tsai, Yi-Ren Yeh, and Yu-Chiang Frank Wang. 2015. Unsupervised domain adaptation with imbalanced cross-domain data. In *Proceedings of the IEEE International Conference on Computer Vision*. 4121–4129. <https://doi.org/10.1109/ICCV.2015.469>
- [17] Anil K Jain, M Narasimha Murty, and Patrick J Flynn. 1999. Data clustering: a review. *ACM computing surveys (CSUR)* 31, 3 (1999), 264–323. <https://doi.org/10.1145/331499.331504>
- [18] JetBrains. 2022. Kotlin Coding Style Guide. <https://developer.android.com/kotlin/style-guide> Accessed: 2022-05-19.
- [19] JetBrains. 2022. Kotlin Programming Language. <https://developer.android.com/kotlin/first> Accessed: 2022-05-11.
- [20] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-aware neural machine translation for automatic program repair. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173. <https://doi.org/10.1109/ICSE43902.2021.00107>
- [21] Heechul Jung, Jeongwoo Ju, Minju Jung, and Junmo Kim. 2016. Less-forgetting learning in deep neural networks. *arXiv preprint arXiv:1607.00122* (2016). <https://doi.org/10.48550/arXiv.1607.00122>
- [22] George Karypis. 2002. *CLUTO—a clustering toolkit*. Technical Report. MINNESOTA UNIV MINNEAPOLIS DEPT OF COMPUTER SCIENCE. <https://hdl.handle.net/11299/215521>
- [23] Jindae Kim and Sunghun Kim. 2019. Automatic patch generation with context-based change application. *Empirical Software Engineering* 24, 6 (2019), 4071–4106. <https://doi.org/10.1007/s10664-019-09742-5>
- [24] Misoo Kim, Youngkyoung Kim, and Eunseok Lee. 2021. A Novel Automatic Query Expansion with Word Embedding for IR-based Bug Localization. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 276–287. <https://doi.org/10.1109/ISSRE52982.2021.00038>
- [25] Serkan Kirbas, Etienne Windels, Olayori McBello, Kevin Kells, Matthew Pagano, Rafal Szalanski, Vesna Nowack, Emily Rowan Winter, Steve Counsell, David Bowes, et al. 2021. On the introduction of automatic program repair in Bloomberg. *IEEE Software* 38, 4 (2021), 43–51. <https://doi.org/10.1109/MS.2021.3071086>
- [26] Gary T Leavens, Alessandro Garcia, Corina S Păsăreanu, Junhee Lee, Seongjoon Hong, and Hakjoo Oh. 2018. MemFix: static analysis-based repair of memory deallocation errors for C. *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (10 2018), 95–106. <https://doi.org/10.1145/3236024.3236079>
- [27] Valentina Lenarduzzi, Francesco Lomio, Heikki Huttunen, and Davide Taibi. 2020. Are sonarqube rules inducing bugs?. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 501–511. <https://doi.org/10.1109/SANER48275.2020.9054821>
- [28] Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. Dlfix: Context-based code transformation learning for automated program repair. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 602–614. <https://doi.org/10.1145/3377811.3380345>
- [29] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 31–42. <https://doi.org/10.1145/3293882.3330577>
- [30] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. AVATAR: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations. *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)* 00 (2 2019), 1–12. <https://doi.org/10.1109/SANER.2019.8667970>
- [31] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshir Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 101–114. <https://doi.org/10.1145/3395363.3397369>
- [32] Diego Marcilio, Rodrigo Bonifácio, Eduardo Monteiro, Edna Canedo, Welder Luz, and Gustavo Pinto. 2019. Are static analysis violations really fixed? a closer look at realistic usage of sonarqube. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 209–219. <https://doi.org/10.1109/ICPC.2019.00040>
- [33] Diego Marcilio, Carlo A. Furia, Rodrigo Bonifácio, and Gustavo Pinto. 2020. SpongeBugs: Automatically generating fix suggestions in response to static code analysis warnings. *Journal of Systems and Software* 168 (10 2020), 110671. <https://doi.org/10.1016/j.jss.2020.110671>
- [34] Alexandru Marginean, Johannes Bader, Satish Chandra, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, and Andrew Scott. 2019. Sappfix: Automated end-to-end repair at scale. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 269–278. <https://doi.org/10.1109/ICSE-SEIP.2019.00039>
- [35] Maurizio Martignano, Andraes Jung, T Ihmann, et al. 2015. Source code analysis of flight software using a SonarQube based code quality platform. *Ada User Journal* 36, 2 (2015), 99.
- [36] Matias Martinez and Bruno Gois Mateus. 2020. Why did developers migrate Android applications from Java to Kotlin? *arXiv preprint arXiv:2003.12730* (2020). <https://doi.org/10.48550/arXiv.2003.12730>
- [37] Ali Mesbah, Andrew Rice, Emily Johnston, Nick Glorioso, and Edward Aftandilian. 2019. Deepdelta: learning to repair compilation errors. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 925–936. <https://doi.org/10.1145/3338906.3340455>
- [38] Keigo Naitou, Akito Tanikado, Shinsuke Matsumoto, Yoshiki Higo, Shinji Kusumoto, Hiroyuki Kirinuki, Toshiyuki Kurabayashi, and Haruto Tanno. 2018. Toward introducing automated program repair techniques to industrial software development. In *2018 IEEE/ACM 26th International Conference on Program Comprehension (ICPC)*. IEEE, 332–3323. <https://doi.org/10.1145/3196321.3196358>
- [39] Kunihiro Noda, Yusuke Nemoto, Keisuke Hotta, Hideo Tanida, and Shinji Kikuchi. 2020. Experience report: How effective is automated program repair for industrial software?. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 612–616. <https://doi.org/10.1109/SANER48275.2020.9054829>
- [40] Andrei V Novikov. 2019. PyClustering: Data mining library. *Journal of Open Source Software* 4, 36 (2019), 1230. <https://doi.org/10.21105/joss.01230>
- [41] Dan Pelleg, Andrew W Moore, et al. 2000. X-means: Extending k-means with efficient estimation of the number of clusters. In *ICML*, Vol. 1. 727–734.
- [42] PMD Open Source Project. 2022. PMD. <https://pmd.github.io/> Accessed: 2022-05-11.
- [43] Payam Refaellizadeh, Lei Tang, and Huan Liu. 2009. Cross-validation. *Encyclopedia of database systems* 5 (2009), 532–538.
- [44] Gregg Rothermel, Doo-Hwan Bae, Hiroaki Yoshida, Rohan Bavishi, Keisuke Hotta, Yusuke Nemoto, Mukul R Prasad, and Shinji Kikuchi. 2020. Phoenix: a tool for automated data-driven synthesis of repairs for static analysis violations. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings* (2020), 53–56. <https://doi.org/10.1145/3377812.3382150>
- [45] SonarSource SA. 2022. SonarQube. <https://www.sonarqube.org/> Accessed: 2022-05-11.
- [46] Nyyti Saarimäki, Maria Teresa Baldassarre, Valentina Lenarduzzi, and Simone Romano. 2019. On the accuracy of sonarqube technical debt remediation time. In

- 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). IEEE, 317–324. <https://doi.org/10.1109/SEAA.2019.00055>
- [47] SpotBugs. 2022. SpotBugs. <https://spotbugs.github.io/> Accessed: 2022-05-11.
- [48] Aparna Growth Strategist. 2019. Kotlin vs Java: Know which is the best programming language for Android App Development in 2021? <https://www.mobileappdaily.com/kotlin-vs-java>
- [49] Chuanqi Tan, Fuchun Sun, Tao Kong, Wenchang Zhang, Chao Yang, and Chunfang Liu. 2018. A survey on deep transfer learning. In *International conference on artificial neural networks*. Springer, 270–279. https://doi.org/10.1007/978-3-030-01424-7_27
- [50] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F Bissyandé. 2020. Evaluating representation learning of code changes for predicting patch correctness in program repair. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 981–992. <https://doi.org/10.1145/3324884.3416532>
- [51] Shangwen Wang, Ming Wen, Bo Lin, Hongjun Wu, Yihao Qin, Deqing Zou, Xiaoguang Mao, and Hai Jin. 2020. Automated patch correctness assessment: How far are we?. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 968–980. <https://doi.org/10.1145/3324884.3416590>
- [52] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-Aware Patch Generation for Better Automated Program Repair. *ICSE* (5 2018). <https://doi.org/10.1145/3180155.3180233>
- [53] Deheng Yang, Kui Liu, Dongsun Kim, Anil Koyuncu, Kisub Kim, Haoye Tian, Yan Lei, Xiaoguang Mao, Jacques Klein, and Tegawendé F Bissyandé. 2021. Where were the repair ingredients for Defects4j bugs? *Empirical Software Engineering* 26, 6 (2021), 1–33. <https://doi.org/10.1007/s10664-021-10003-7>
- [54] Michihiro Yasunaga and Percy Liang. 2020. Graph-based, self-supervised program repair from diagnostic feedback. In *International Conference on Machine Learning*. PMLR, 10799–10808.
- [55] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 341–353. <https://doi.org/10.1145/3468264.3468544>
- [56] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. 2020. A comprehensive survey on transfer learning. *Proc. IEEE* 109, 1 (2020), 43–76. <https://doi.org/10.1109/JPROC.2020.3004555>